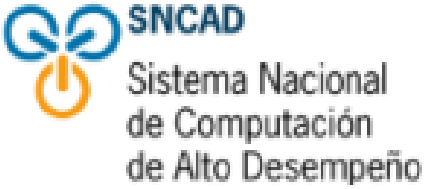
Introducción a PyCuda

Clase 6

Pablo Cappagli

pablocappagli@gmail.com



https://documen.tician.de/pycuda/index.html

pycuda.compiler.SourceModule()

class pycuda.compiler.**SourceModule**(source, nvcc="nvcc", options=None, keep=False, no_extern_c=False, arch=None, code=None, cache_dir=None, include_dirs=[])

ejemplo_double.py

```python
import numpy as np
import pycuda.driver as cuda
import pycuda.autoinit
from pycuda.compiler import SourceModule

mod = SourceModule("""
__global__ void times_two(int N, float *a)
{
   int id = blockIdx.x*blockDim.x + threadIdx.x;
   if(id<N){
       a[id] = 2*a[id];
   }
}
""")
```

```
N = 30000

a = np.ones(N).astype(np.float32)

a_gpu = cuda.mem_alloc(a.nbytes)
cuda.memcpy_htod(a_gpu, a)

func = mod.get_function('times_two')

numThreads = 128
numBlocks = int((N + numThreads - 1 )/numThreads)

func(np.array(N), a_gpu, block=(numThreads,1,1), grid=(numBlocks,1,1))

a_doubled = np.empty_like(a)
cuda.memcpy_dtoh(a_doubled, a_gpu)


print(a_doubled)
print(a)
```

Transferencia de memoria simplificada:

- pycuda.driver.In()
- pycuda.driver.Out()
- pycuda.driver.InOut()

ejemplo_double_inout.py

```
import numpy as np
import pycuda.driver as cuda
import pycuda.autoinit
from pycuda.compiler import SourceModule

mod = SourceModule("""
__global__ void times_two(int N, float *a)
{
   int id = blockIdx.x*blockDim.x + threadIdx.x;
   if(id<N){
        a[id] = 2*a[id];
   }
}
""")
```

```
N = 30000

a = np.ones(N)
a = a.astype(np.float32)

func = mod.get_function('times_two')

numThreads = 128
numBlocks = (N + numThreads - 1 )//numThreads

func(np.array(N), cuda.InOut(a), block=(numThreads,1,1),
   grid=(numBlocks,1,1))

print(a)
```

ejemplo_hello_gpu.py

```python
import numpy as np
import pycuda.driver as cuda
import pycuda.autoinit
from pycuda.compiler import SourceModule

mod = SourceModule("""
__global__ void multiply_them(float *dest, float *a, float *b)
{
   int i = threadIdx.x;
   dest[i] = a[i] * b[i];
}
""")
multiply_them = mod.get_function("multiply_them")

a = np.random.randn(400).astype(np.float32)
b = np.random.randn(400).astype(np.float32)

dest = np.zeros_like(a)
multiply_them(cuda.Out(dest), cuda.In(a), cuda.In(b), block=(400,1,1))
print(dest-a*b)
```

pycuda.gpuarray.GPUArray

ejemplo_gpu_array.py

```
import pycuda.gpuarray as gpuarray
import pycuda.driver as cuda
import pycuda.autoinit

import numpy as np

my_gpu_array = gpuarray.GPUArray([5,5], dtype = np.float32)

print(my_gpu_array)
print(my_gpu_array.dtype)
print(my_gpu_array.shape)
print(my_gpu_array.size)
```

ejemplo_gpu_array_02.py

```
import pycuda.gpuarray as gpuarray
import pycuda.driver as cuda
import pycuda.autoinit
import numpy as np

a_gpu = gpuarray.GPUArray([50,60], dtype = np.float32)
a_gpu.fill(3.14)

a = np.ones([50,60]).astype(np.float32)*3.14
a_gpu = gpuarray.to_gpu(a) #<- copia a GPU

a_gpu = gpuarray.empty([50,60], dtype = np.float32)
a_gpu = gpuarray.zeros([50,60], dtype = np.float32)
# d_gpu = gpuarray.ones([100,120], dtype = np.float32) # <- no existe

b_gpu = gpuarray.empty_like(a_gpu)
b_gpu = gpuarray.zeros_like(a_gpu)
b_gpu = gpuarray.ones_like(a_gpu)

c_gpu = gpuarray.arange(0, 60, dtype = np.float32)
print(c_gpu)
```

ejemplo_gpu_array_03.py

```
import pycuda.gpuarray as gpuarray
import pycuda.driver as cuda
import pycuda.autoinit
import numpy as np

a_gpu = gpuarray.GPUArray([50,60], dtype = np.float32)
a_gpu.fill(3.14)

b_gpu = gpuarray.arange(0, 3000, dtype = np.float32)


print(gpuarray.sum(a_gpu))

print(gpuarray.dot(a_gpu, b_gpu))

print(gpuarray.max(b_gpu))

print(gpuarray.min(b_gpu))
```

ejemplo_gpu_array_04.py

```
import pycuda.gpuarray as gpuarray
import pycuda.driver as cuda
import pycuda.autoinit
import numpy as np
import pycuda.cumath as cumath

a_gpu = gpuarray.GPUArray([50,60], dtype = np.float32)
a_gpu.fill(144.0)

i_gpu = cumath.fabs(a_gpu)
j_gpu = cumath.floor(a_gpu)
k_gpu = cumath.exp(a_gpu)
l_gpu = cumath.log(a_gpu)
m_gpu = cumath.sqrt(a_gpu)
p_gpu = cumath.sin(a_gpu)
q_gpu = cumath.tan(a_gpu)

print(m_gpu)
```

ejemplo_double_simple.py

```
import pycuda.gpuarray as gpuarray
import pycuda.driver as cuda
import pycuda.autoinit

import numpy as np

N = 30000

a = np.ones(N)
a = a.astype(np.float32)

a_gpu = gpuarray.to_gpu(a)
a_doubled = (2*a_gpu).get()

print(a)
print(a_doubled)
```

pycuda.elementwise.ElementwiseKernel()

class pycuda.elementwise.**ElementwiseKernel**(*arguments, operation, name="kernel", keep=False, options=[], preamble=""*)

ejemplo_elementwise.py

```
import pycuda.gpuarray as gpuarray
import pycuda.autoinit
from pycuda.elementwise import ElementwiseKernel
from pycuda.curandom import rand as curand
import numpy as np

a_gpu = curand((50,))
b_gpu = curand((50,))

lin_comb = ElementwiseKernel(
    "float a, float *x, float b, float *y, float *z",
        "z[i] = a*x[i] + b*y[i]",
            "linear_combination" )


c_gpu = gpuarray.empty_like(a_gpu)
lin_comb(5, a_gpu, 6, b_gpu, c_gpu)

print(np.all((c_gpu - (5*a_gpu+6*b_gpu)).get() < 1e-6))
```

ejemplo_elementwise_02.py

```python
import pycuda.gpuarray as gpuarray
import pycuda.autoinit
from pycuda.elementwise import ElementwiseKernel
from pycuda.curandom import rand as curand
import numpy as np
import pycuda.cumath as cumath

lin_comb = ElementwiseKernel(
        "float a, float *x, float b, float *y, float *z",
        "z[i] = my_f(a*x[i], b*y[i])",
        "linear_combination",
        preamble="""
        __device__ float my_f(float x, float y)
        {
          return sin(x*y);
        }
        """)
```

```
a_gpu = curand((50,))
b_gpu = curand((50,))

c_gpu = gpuarray.empty_like(a_gpu)
lin_comb(5, a_gpu, 6, b_gpu, c_gpu)

print(np.all((c_gpu - cumath.sin(5*a_gpu*6*b_gpu)).get() < 1e-5))
```

pycuda.reduction.ReductionKernel()

class pycuda.reduction.**ReductionKernel**(dtype_out, neutral, reduce_expr, map_expr=None, arguments=None, name="reduce_kernel", keep=False, options=[], preamble="", allocator=None)

reduction.py

```
import pycuda.gpuarray as gpuarray
import pycuda.driver as cuda
import pycuda.autoinit
from pycuda.reduction import ReductionKernel
import numpy as np

a = gpuarray.arange(400, dtype=np.float32)
b = gpuarray.arange(400, dtype=np.float32)

krnl = ReductionKernel(np.float32, neutral="0",
       reduce_expr="a+b", map_expr= "x[i]*y[i]" ,
       arguments="float *x, float *y")

my_dot_prod = krnl(a, b).get()
print(my_dot_prod)
print( (my_dot_prod - gpuarray.sum(a*b).get()) < 1e-6)
```

GPGPU Algorithms

Bogdan Opanchuk's **reikna** offers a variety of GPU-based algorithms (FFT, RNG, matrix multiplication) designed to work with pycuda.gpuarray.GPUArray objects.

https://pypi.org/project/reikna/

Reikna is a library containing various GPU algorithms built on top of PyCUDA and PyOpenCL. The main design goals are:

- separation of computation cores (matrix multiplication, random numbers generation etc) from simple transformations on their input and output values (scaling, typecast etc);
- separation of the preparation and execution stage, maximizing the performance of the execution stage at the expense of the preparation stage (in other words, aiming at large simulations)
- partial abstraction from CUDA/OpenCL